

Exercise 2.2 — GitHub Actions CI Workflow for Terraform

Course: Optimizaciones y Desempeño — Cloud Deployment Automation

Session: 2 — April 30, 2026

Time allowed: 30 minutes

Submission: Initialize a new repository called **oyd-exercise-2-2** and commit/push everything into it. **Submit the repository URL only.**

Context

The following Terraform workspace provisions an S3 bucket. Copy these files into your repository before writing the workflow — do not modify them.

provider.tf

```
terraform {
  required_version = ">= 1.8"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}
```

variables.tf

```
variable "region" {  
  description = "AWS region to deploy resources in"  
  type        = string  
  default     = "us-east-1"  
}  
  
variable "environment" {  
  description = "Deployment environment"  
  type        = string  
  default     = "dev"  
}  
  
variable "bucket_name_prefix" {  
  description = "Prefix for the S3 bucket name"  
  type        = string  
}
```

main.tf

```
resource "aws_s3_bucket" "main" {  
  bucket = "${var.bucket_name_prefix}-${var.environment}"  
}
```

outputs.tf

```
output "bucket_name" {  
  description = "Name of the provisioned S3 bucket"  
  value       = aws_s3_bucket.main.bucket  
}
```

envs/dev/dev.tfvars

```
region          = "us-east-1"  
environment     = "dev"  
bucket_name_prefix = "oyd-ex2"
```

The CI pipeline will validate every pull request targeting main and post the full Terraform plan as a collapsible comment on the PR.

Setup

- A GitHub account with a repository to push to
- AWS credentials set as repository secrets: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION`
- No local Terraform installation required — all steps run inside the GitHub Actions runner

Tasks

Task 1 — Workflow file and trigger

1. Create `.github/workflows/terraform-ci.yml`.
2. Trigger on pull requests targeting `main` only.
3. Declare a `permissions` block with contents: `read` and `pull-requests: write`.

Task 2 — Validation steps

1. Install Terraform on the runner using `hashicorp/setup-terraform@v3`.
2. Add a step running `terraform fmt --check -recursive`. Must fail the PR on non-zero exit. No `continue-on-error`.
3. Add a step running `terraform init -backend=false`.
4. Add a step running `terraform validate`.

Task 3 — Plan step

1. Configure AWS credentials using `aws-actions/configure-aws-credentials@v4`, sourcing all values exclusively from repository secrets.
2. Add a step running `terraform plan -var-file=envs/dev/dev.tfvars`, capturing the full output to `plan.txt` using `tee`. Give this step an `id`.

Task 4 — Plan as PR comment

1. Add a step that reads `plan.txt` and posts it as a PR comment using `actions/github-script@v7`.

2. Wrap the output in a collapsible `<details>` block with summary text Terraform Plan.
3. This step may use `continue-on-error: true` — a failure to post the comment must not block the PR.

Task 5 — README

1. Create `README.md` at the repository root with a `## Evidence` section containing:
 - A direct link to the pull request that triggered the successful pipeline run
 - A screenshot of the PR comment showing the Terraform plan output, saved as `evidence/pr-comment.png`, rendered inline: `![PR comment](evidence/pr-comment.png)`

Acceptance Criteria

- Workflow file exists at `.github/workflows/terraform-ci.yml`
- Triggers on PRs to `main` only; `permissions` block declares `pull-requests: write`
- Steps execute in order: `fmt` → `init` → `validate` → `plan` → `post comment`
- No credentials, region names, or account IDs hardcoded in any YAML or `.tf` file
- `continue-on-error: true` absent from all steps except optionally the `comment` step
- A PR with a formatting error causes the workflow to fail and block the merge
- PR comment contains the full plan output in a collapsible `<details>` section
- `README.md` renders the PR comment screenshot inline and includes the PR link under Evidence